

Universidad Galileo

Postgrado en Diseño y Desarrollo de Software
Infraestructura en la Nube — Ciclo Mayo–Junio 2026

PROYECTO INFRAESTRUCTURA EN LA NUBE

Sistema de Procesamiento de Archivos de Ventas y Generación de Reportes

Equipo:

Gabriela Lucia Navarro de León - 20000127

Diego Alejandro Sican Olivares - 19001690

Sandra Daniela Soria Palma - 20002619

Fecha de entrega: 21 de junio de 2026

Opción elegida: Proyecto 4 — Sistema de procesamiento de archivos y reportes

1. Resumen de Cambios

Para esta entrega se consolida el documento final de diseño del SPVR incorporando las decisiones de seguridad, observabilidad, costos, riesgos e integración. La arquitectura definida en las entregas anteriores se mantiene como base, pero ahora se completa con controles de acceso, roles IAM por servicio, secretos gestionados, cifrado, métricas, alarmas y un plan de despliegue automatizado.

Se realizaron algunos cambios respecto a las entregas anteriores, se actualizó la tabla de usuarios incorporando el campo `password_hash`, el cual es necesario para el soporte de autenticación, además, se agregó una nueva tabla para poder almacenar los reportes individuales de los vendedores y visualizar su desempeño.

Las demás entregas se mantienen sin cambios tomando en cuenta que se quiere pasar de una arquitectura funcionalmente asíncrona documentada de forma integral. Lambda API, Lambda Worker, Lambda Cleanup, SQS, DLQ, RDS, S3 y SES continúan cumpliendo sus responsabilidades, pero ahora se especifica qué permisos necesita cada componente, qué secretos consume, cómo se protegen los datos y cómo se monitorea el comportamiento operativo del sistema.

También se incorpora autenticación real para el portal mediante contraseñas hasheadas con `bcrypt` y tokens JWT con expiración. Además, el sistema puede identificar al usuario que ha sido autenticado, validando y bloqueando los roles correspondientes. El analista de ventas puede consultar los reportes generados asociados a él mismo, mientras que los otros roles tienen acceso según sus funciones.

Finalmente, se definen alarmas de CloudWatch, dashboard operativo, presupuesto mensual, federación OIDC para GitHub Actions, deployment one-click, riesgos principales y el detalle del componente más complejo: el pipeline de procesamiento de archivos CSV.

2. Resumen Ejecutivo

Las empresas generan periódicamente registros de ventas a través de sus sistemas de facturación, puntos de venta o herramientas como Excel. Sin embargo, transformar esta información en reportes útiles para apoyar la toma de decisiones suele ser un proceso manual, el cual es realizado por el analista de ventas, lo que puede ocasionar que este sea propenso a errores humanos y al uso ineficiente del tiempo.

El Sistema de Procesamiento de Ventas y Reportes (SPVR) tiene como objetivo automatizar este proceso. Para ello, el analista tendrá que cargar el archivo CSV en el portal y el sistema se encargará de procesarlo en segundo plano. Como resultado, se generará un reporte PDF el cual contendrá el análisis de ventas por producto, ciudad, cliente y por periodo. Una vez finalizado el proceso, se le notificará al analista por medio de un correo electrónico que ya puede descargar el reporte. Además, el gerente de ventas puede consultar en cualquier momento el historial de reportes generados. Por otra parte, los vendedores tienen acceso a un resumen personalizado de su desempeño en el período actual, que muestra el total vendido, la cantidad de productos comercializados, los clientes atendidos y su posición dentro del ranking general de ventas, junto con el detalle de los productos más vendidos y la evolución mensual de sus resultados, siempre que el archivo haya sido previamente procesado por el analista.

Con esta automatización, el sistema evita errores al consolidar la información, reduce los retrasos en las entregas de los reportes, mantiene un historial de archivos procesados y elimina la necesidad de que el analista distribuya manualmente los resultados a cada vendedor.

3. Actores del Sistema

3.1 Actores Primarios

Son aquellos que inician las acciones principales del sistema. Sin ellos, el sistema no hace nada.

- **Analista de Ventas:** Sube los archivos CSV de las ventas, consulta el estado de sus trabajos y descarga los reportes generados.
- **Gerente de Ventas:** Consulta el historial de reportes y los descarga para presentar a la dirección o define estrategias para aumentar las ventas.

3.2 Actores de Soporte

Son aquellos que interactúan con el sistema de forma secundaria en donde consumen resultados, supervisan su funcionamiento o verifican su uso, pero no forman parte del flujo principal.

- **Vendedor:** Consulta y descarga su propio reporte de desempeño.
- **Administrador:** Gestiona a los usuarios y sus roles, monitorea el sistema y revisa los logs de errores.
- **Auditor:** Consulta y descarga cualquier reporte que fue generado por el sistema, para verificar que los datos sean consistentes y no hayan sido alterados. No puede modificar nada.

4. Casos de Uso Priorizados

Se presentan 8 user stories priorizadas según su importancia para el funcionamiento del sistema:

Prioridad	User Story	Criterio de Éxito
-----------	------------	-------------------

P0	Como analista, quiero cargar un archivo CSV de ventas desde el portal para que el sistema lo procese automáticamente.	El archivo se sube correctamente y el estado cambia a PENDIENTE en menos de 5 segundos.
P0	Como analista, quiero recibir una notificación por correo cuando mi reporte PDF esté listo para poder descargarlo.	El correo llega al finalizar el procesamiento con un enlace directo al reporte.
P0	Como analista, quiero descargar el reporte PDF generado para compartirlo con el gerente de ventas.	El reporte descargado contiene los análisis correctos del CSV cargado.
P1	Como gerente de ventas, quiero consultar el historial de reportes generados para comparar resultados entre períodos.	La lista muestra fecha, ID, nombre del archivo, acciones y estado de cada reporte anterior.
P1	Como analista, quiero ver el estado de mi trabajo en tiempo real para saber si necesito esperar.	El estado se actualiza mostrando PENDIENTE, PROCESANDO, COMPLETADO o FALLIDO sin recargar la página.
P1	Como vendedor, quiero revisar mi propio resumen de ventas para conocer mi desempeño del período.	El reporte muestra total vendido, productos vendidos, los clientes atendidos, la posición en el ranking, el detalle de los productos más vendidos y la evolución mensual de sus resultados. No tienen acceso a datos de otros vendedores.
P2	Como administrador, quiero tener visibilidad de los errores ocurridos en el sistema para garantizar su correcto funcionamiento.	El sistema presenta un registro de fallos con la información necesaria para identificar qué ocurrió y cuándo.
P2	Como auditor, quiero consultar y descargar cualquier reporte PDF generado en el sistema para verificar que los datos sean correctos y consistentes.	El sistema permite ver y descargar el reporte PDF de cualquier usuario. No tiene la opción de modificar nada.

5. Funcionalidades Específicas del Proyecto

Las siguientes funcionalidades que diferencian a este sistema de otros similares:

Funcionalidad	Descripción
Validación de estructura del CSV de ventas	El sistema verifica las siguientes columnas: sale_id, sale_date, product_name, quantity, unit_price, city, salesperson_name, entre otras. Rechaza cualquier archivo que no cumplan la estructura.
Reporte de productos más vendidos	Genera un ranking de los top 10 productos más vendidos y cuanto representan del total de ingresos.
Análisis de clientes frecuentes	Identifica a los clientes con mayor número de compras en el período usando customer_id y customer_name.

Ranking de ventas por ciudad	Compara el desempeño de ventas entre ciudades usando la columna city del CSV.
Análisis de ventas mensuales	Agrupar las ventas por mes usando sale_date para identificar en que meses se vendió más.
Reporte individual por vendedor	Genera un reporte personalizado por salesperson_id que incluya: total vendido en el período, cantidad de productos vendidos, número de clientes atendidos, posición en el ranking general de ventas, una lista detallada de productos más vendidos y su evolución mensual.
Identificador único por carga	Cada archivo CSV recibe un ID único para rastrear su estado y hacer referencia a los reportes que fueron generados.
Notificación automática al analista	El sistema envía un correo al analista con un enlace directo al reporte PDF generado.
Registro de errores del sistema	El sistema guarda un registro de errores por cada trabajo que no pudo procesarse, incluyendo job_id, fecha, usuario y descripción del error. El administrador puede consultarlo desde el portal.
Acceso de auditoría a reportes	El auditor puede consultar y descargar cualquier reporte PDF que fue generado por cualquier usuario desde el portal, sin importar quién lo solicitó. No puede modificar ni eliminar nada.
Descarga de reportes por período	El gerente de ventas puede filtrar el historial de reportes por período y descargar el PDF correspondiente directamente desde el portal para presentarlo en reuniones de dirección.

6. Mockups del Frontend

Se presentan 8 pantallas low-fi que cubren los casos de uso priorizados. Los borradores generados por IA y los mockups finales editados por el equipo se encuentran en el Anexo de Mockups al final de este documento.

Pantalla 1 — Login

Caso de uso: Acceso al sistema. Contiene un formulario con un campo de correo electrónico y contraseña. Al autenticarse, el sistema debe de identificar el rol del usuario y debe de redirigirlo a su vista correspondiente.

Pantalla 2 — Dashboard principal

Caso de uso: P0 — El analista consulta el estado de sus trabajos recientes. Se muestran 3 métricas: trabajos completados, en proceso y con error. Se incluye una lista de trabajos con nombre del archivo, ID único, fecha, estado actual y acciones.

Pantalla 3 — Carga de archivo CSV

Caso de uso: P0 — El analista sube archivo CSV de ventas para procesamiento. Se muestra la zona en donde se puede arrastrar el archivo o seleccionarlo desde su computadora. La pantalla muestra las columnas que debe tener el archivo. Si el archivo no tiene la estructura correcta, el sistema muestra un mensaje indicando qué columna falta o está incorrecta.

Pantalla 4 — Estado del trabajo en tiempo real

Caso de uso: P1 — El analista monitorea el progreso de su trabajo sin recargar la página. Se muestra el nombre del archivo y su ID único, junto con los pasos: archivo recibido, validación, procesando, generando PDF y notificación enviada.

Pantalla 5 — Historial de reportes

Caso de uso: P1 — El gerente consulta y descarga reportes anteriores filtrados por período. El auditor accede a esta misma vista con la opción de descarga habilitada para que pueda verificar que los datos sean correctos y consistentes, pero no tiene la opción de modificar los archivos.

Pantalla 6 — Previsualización del reporte PDF

Caso de uso: P0 — El analista y el gerente visualizan y descargan el reporte generado. Se muestran las métricas principales: total vendido, producto más vendido y ciudad con más ventas, junto con el listado de productos, ciudades y clientes frecuentes.

Pantalla 7 — Resumen del vendedor

Caso de uso: P1 — El vendedor consulta únicamente su propio desempeño. Se muestra el total vendido, el número de clientes atendidos, los productos vendidos, su posición en el ranking general y la evolución de sus ventas mes a mes. No tiene acceso a datos de otros vendedores.

Pantalla 8 — Registro de errores

Caso de uso: P2 — El administrador consulta los trabajos que fallaron durante el procesamiento. Se muestra el ID del trabajo, el usuario que lo solicitó, la fecha y la descripción del error para identificar qué ocurrió y cuándo.

7. Mapeo a Conceptos del Curso

La siguiente tabla conecta las funcionalidades del sistema con los componentes del curso que se ejercitarán:

Componente del Curso	Cómo lo ejercita este proyecto
Cómputo (API)	El API Gateway recibe las solicitudes del frontend y las dirige hacia una Lambda API la cual se encargará de la autenticación, la carga de archivos CSV, consultar los estados y el historial de reportes. Otra lambda Worker se encargará del procesamiento de archivos CSV y generará los reportes PDF.
Base de datos	Se almacena la información de cada trabajo subido por el analista y los reportes generados. También incluye el registro de errores para que el administrador pueda consultarlo.
Almacenamiento de archivos	Se utilizarán dos buckets de almacenamiento, los cuales guardarán los archivos CSV y los reportes PDF generados para el vendedor y en general.
Procesamiento asíncrono	Cuando se sube el archivo CSV, el sistema publicará un mensaje en SQS. El worker corriendo en segundo plano tomará ese mensaje, lo procesará y generará el reporte PDF. Cuando se finaliza el proceso, se enviará una notificación por correo al analista con el enlace para descargar el reporte.
Red (conceptual)	El frontend y API Gateway forman parte de la capa pública del sistema. Las funciones Lambda API y Lambda Worker se ejecutarán dentro de las subnets privadas junto a la base de datos RDS MySQL. El acceso saliente se realizará mediante NAT Gateway y la comunicación interna se controlará mediante los security groups.
Seguridad	El acceso es controlado por roles: el analista sube archivos y descarga reportes, el gerente consulta y descarga, el vendedor solo ve su reporte personal, el auditor descarga sin poder modificar nada y el administrador gestiona usuarios y roles.
Observabilidad	El sistema registra los logs de errores para que el administrador los consulte, monitorea trabajos en cola y tiempo de procesamiento, y activa una alarma cuando los trabajos fallidos superan un límite definido.

8. Scope del Sistema

8.1 Lo que SÍ hace el sistema

- Recibir y validar los archivos CSV de ventas con la estructura definida.
- Procesar los datos de forma asíncrona y generar un reporte PDF con el análisis de ventas por producto, ciudad, cliente y período.
- Generar un reporte personalizado por cada vendedor con su desempeño.
- Notificar al analista por correo que el reporte está finalizado.
- Historial de reportes generados por usuario.
- Controlar el acceso al sistema según el rol de cada actor.
- Reintentar el procesamiento automáticamente en caso de fallo.

8.2 Lo que NO hace el sistema (fuera de scope)

- No procesa archivos PDF ni imágenes como entrada, solo CSV.
- No calcula comisiones ni márgenes de ganancia.
- No se conecta a sistemas externos de facturación o ERPs.
- No genera análisis en tiempo real.
- No permite el registro propio de usuarios, los accesos son gestionados por el administrador.
- No registra ventas individuales, el archivo CSV es preparado por el analista desde fuentes externas como Excel o sistemas de facturación.

9. Diagrama de contexto

El diagrama de contexto presenta el SPVR como una sola caja, mostrando quién interactúa con él y de qué servicios externos depende. Los componentes internos del sistema, como el portal, la API, la base de datos y el almacenamiento no se detallan todavía en este nivel; se explicarán con mayor detalle en el diagrama de contenedores en la Entrega 3.

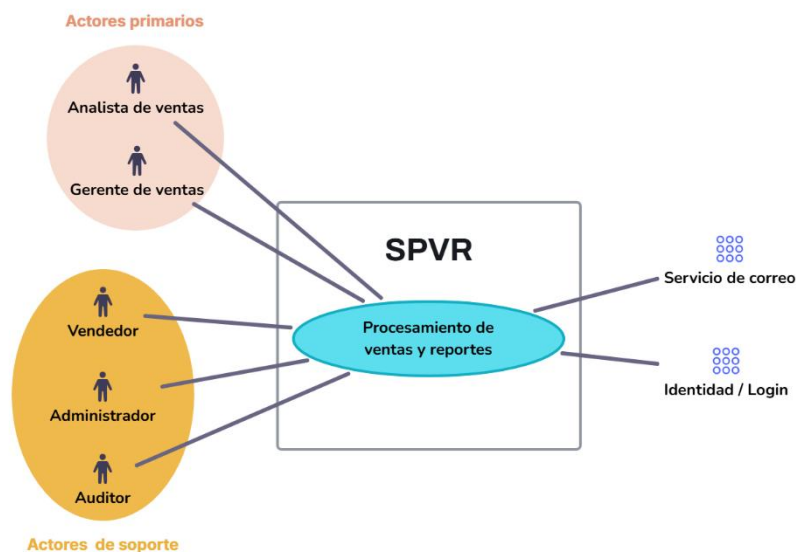


Figura 1. Diagrama de contexto del SPVR.

Los actores se dividen en primarios y de soporte. Como servicios externos se identifican el servicio de correo, que notifica al analista cuando un reporte está listo, y un proveedor de identidad para el login por rol, cuya tecnología concreta queda como decisión pendiente para entregas posteriores. Los servicios de AWS no se consideran sistemas externos, ya que son las piezas internas con las que se construye el sistema.

10. Decisión de cómputo

Enfoque elegido: AWS Lambda (serverless).

El sistema maneja dos tipos de tareas. Las primeras corresponden a acciones realizadas por el usuario, como subir archivos CSV, consultar el estado de un proceso o ver el historial de reportes. Estas operaciones serán manejadas por una Lambda API que será expuesta mediante la API Gateway. La segunda se encargará del procesamiento del CSV: validar su estructura, calcular las métricas y generar los reportes en PDF. Estas tareas serán ejecutadas por una Lambda Worker independiente.

Se eligió AWS Lambda para ambas partes. La Lambda Worker se configuró con 512 MB de memoria, ya que procesar un CSV de miles de registros y generar el PDF correspondiente requiere más recursos que una configuración mínima. Además, se definió un timeout de 30 segundos, suficiente para los archivos previstos en esta etapa. El timeout es un parámetro configurable que puede aumentarse si las pruebas muestran que este valor se queda corto con archivos más grandes o con reportes que incluyan muchos gráficos.

Trade-offs considerados

- **Lambda vs. EC2.** EC2 ofrece mayor control sobre el entorno de ejecución y no tendría límite de tiempo, pero obligaría a mantener los servidores encendidos y generando costos, aunque no haya archivos que procesar. Dado que el procesamiento del SPVR ocurre solo cuando haya archivos que procesar, no resulta conveniente pagar por recursos que estarán inactivos la mayor parte del tiempo. Por ello, Lambda se ajusta mejor al proyecto, ya que solo se pagará por el tiempo real de ejecución.
- **Lambda vs. Fargate.** Fargate evita administrar servidores y no tiene el límite de tiempo de ejecución de Lambda, lo cual sería útil para procesar archivos CSV muy grandes. Sin embargo, utiliza contenedores que pueden tardar más en iniciar y mantener recursos activos. En cambio, Lambda escala a tareas más cortas ya que se ejecuta automáticamente cuando ocurre un evento, como la carga de un archivo. Además, se integra fácilmente con los servicios de AWS como S3, SQS y SES.

Desventaja reconocida

Lambda tiene un límite de 15 minutos por ejecución y un tope de memoria por función. Si un archivo CSV fuera excepcionalmente grande, el procesamiento podría no terminar dentro de ese límite. El equipo asume que los archivos de ventas se mantienen en un rango manejable. Si ese rango llega a superarse, se tomará la decisión de migrar hacia otro servicio, donde solo se moverá el trabajador a Fargate, conservando la API en Lambda.

11. Modelo de datos

Almacenamiento: Amazon RDS con MySQL para la metadata operativa, Amazon S3 para los archivos.

El sistema maneja dos tipos de datos. Por un lado, la información relacionada con el funcionamiento del sistema, como quién subió un archivo, el estado de cada uno de los procesos, los reportes generados y errores acontecidos. La información se guarda principalmente en una base de datos relacional usando Amazon RDS con MySQL.

Por otro lado, los archivos grandes, como los CSV de entrada y los reportes PDF que fueron generados, se almacenan en Amazon S3. Los datos de las ventas contenidos en el CSV no se guardan en la DB; el sistema inicialmente los lee desde S3, procesa la información temporalmente y luego genera el reporte en PDF.

Por qué MySQL y no DynamoDB

El sistema maneja datos con relaciones claras entre usuarios, trabajos, reportes y errores, por lo que requiere consultas como obtener los trabajos de un usuario, filtrar reportes por período y registrar errores durante el procesamiento. Estos tipos de consultas se adaptan mejor en un modelo relacional en SQL; en DynamoDB se requeriría realizar múltiples índices secundarios globales (GSI), lo que aumentaría la complejidad del modelo y podría generar costos adicionales. Además, el equipo ya cuenta con experiencia previa trabajando con MySQL, lo que facilita el diseño y el mantenimiento.

Esquema de la base de datos

El modelo se compone de cuatro tablas. La tabla de trabajos es la entidad central que rastrea cada carga de archivo.

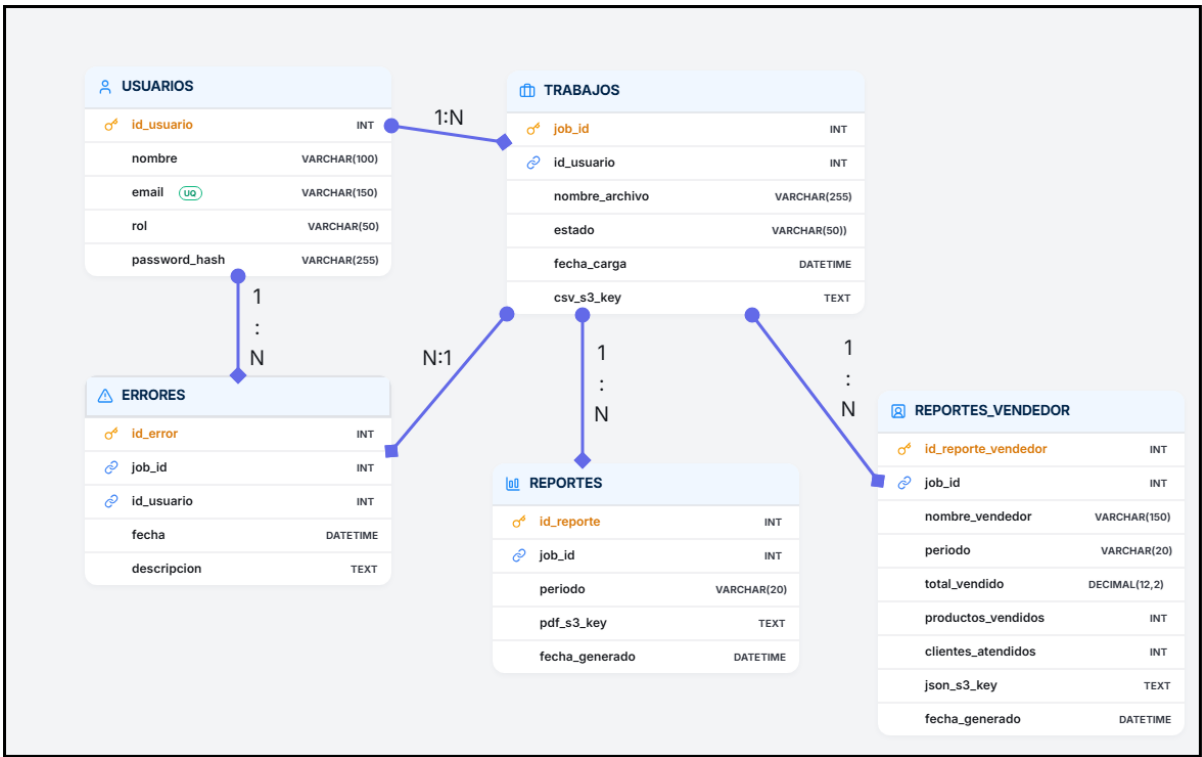


Figura 2. Modelo entidad-relación de la base de datos (RDS MySQL).

Tabla	Contenido	Propósito
-------	-----------	-----------

usuarios	id, nombre, email, rol, password_hash	Registra a cada actor del sistema con su rol y el hash bcrypt de su contraseña.
trabajos	job_id, id_usuario, nombre_archivo, estado, fecha_carga, csv_s3_key	Tabla principal; rastrea cada carga de CSV y su estado.
reportes	id_reporte, job_id, periodo, pdf_s3_key, fecha_generado	Referencia al PDF generado para cada trabajo.
errores	id_error, job_id, id_usuario, fecha, descripcion	Registro de fallos que consulta el administrador.
reportes_vendedor	id_reporte_vendedor, job_id, nombre_vendedor, periodo, total_vendido, productos_vendidos, clientes_atendidos, json_s3_key, fecha_generado	Referencia al JSON de detalle almacenado en S3 para cada reporte.

Las tablas guardan las rutas a los archivos.

Patrones de acceso principales

1. Listar los trabajos de un usuario y su estado actual (dashboard del analista).
2. Filtrar el historial de reportes por período (gerente y auditor).
3. Consultar los errores de procesamiento ordenados por fecha (administrador).

Qué va en S3 vs. base de datos

Se utilizan dos buckets de S3: uno para los CSV que sube el analista y otro para los PDF generados. Separarlos permite aplicar permisos y políticas de ciclo de vida distintas a cada tipo de archivo. La base de datos solo guarda la ruta de cada archivo, manteniendo separados los archivos grandes de la información que el sistema necesita consultar.

Decisión de caché

Para esta etapa no se incluye una capa de caché, ya que el volumen de consultas es bajo y corresponde a una cantidad mínima de usuarios. Con esta carga RDS responde adecuadamente sin necesidad de una capa adicional. En este momento agregar caché aumenta la complejidad y no tendría beneficio. Si en el futuro el historial de reportes recibiera muchas lecturas concurrentes, se evaluaría una caché para esas consultas.

12. Diagrama de contenedores v1

El diagrama de contenedores muestra las ubicaciones de los componentes principales dentro de la red. Para entrega se priorizo la separación entre la capa pública y privada. La integración asíncrona completa se mantiene pendiente ya que se completará en la E4.

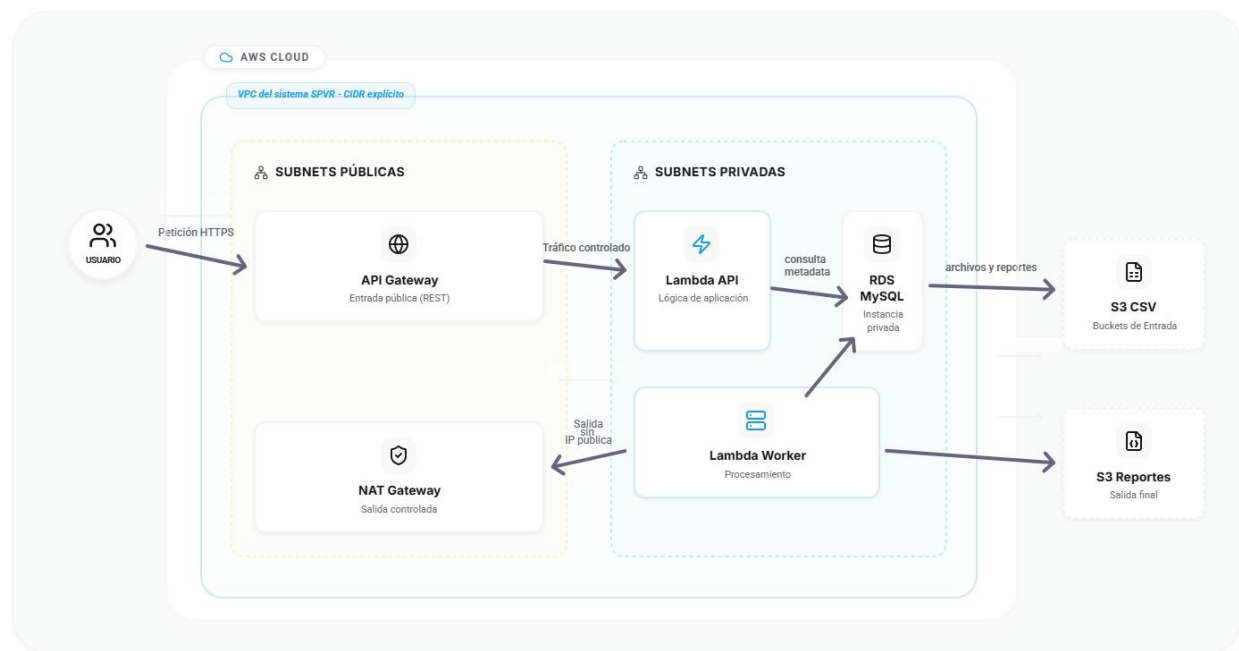


Figura 3. Diagrama de contenedores v1 con separación de red pública y privada.

Contenedor / recurso	Ubicación	Justificación
API Gateway / entrada pública	Capa pública	Recibe tráfico de usuarios y expone la entrada controlada del sistema.
Lambda API	Subnets privadas	Ejecuta lógica de aplicación y accede a RDS sin exponer la base de datos.
Lambda Worker	Subnets privadas	Procesa tareas internas y puede acceder a datos privados.
RDS MySQL	Subnets privadas	Guarda información operativa y no debe recibir tráfico directo desde internet.
S3 CSV y reportes	Servicio administrado AWS	Almacena archivos grandes; el acceso se controla con permisos IAM.
NAT Gateway	Subnet pública	Permite salida desde subnets privadas sin asignar IP pública a los recursos internos.

13. Diseño de red

13.1 Diseño general de la VPC

Para la arquitectura de red, se usará una VPC con el bloque CIDR 10.0.0.0/16. Inicialmente el rango fue elegido porque ofrece suficiente capacidad para dividir la red en distintas subredes sin limitar el crecimiento del proyecto desde las primeras etapas. Además, la solución no requiere

una gran cantidad de subnets, el diseño permite incorporar en el futuro nuevos servicios, ambientes o componentes internos sin tener la necesidad de rediseñar la red principal completa.

La solución será desplegada en dos AZ's dentro de la región us-east-1. Esto permite distribuir los recursos en más de una zona de disponibilidad, reduciendo el impacto ante posibles fallos en una de ellas. En esta fase no se plantea una arquitectura completamente orientada a alta disponibilidad, pero se establece una base más ordenada, escalable y preparada para mejoras a futuro.

Además, la VPC separará los recursos en subredes públicas y privadas. Esta separación permite mantener a los componentes que necesitan recibir tráfico externo, mientras que los servicios internos permanecen protegidos. En este caso, Amazon RDS con MySQL se ubicará en subnets privadas, ya que la base de datos no debe tener acceso directo desde internet. Entonces los componentes internos como las funciones Lambda funcionarán dentro de la capa privada, comunicándose con la base de datos de forma más segura.

13.2 Distribución de subnets

Subnet	Tipo	CIDR	Availability Zone	Componentes
Public Subnet A	Pública	10.0.1.0/24	us-east-1a	NAT Gateway
Public Subnet B	Pública	10.0.2.0/24	us-east-1b	Recursos públicos adicionales para expansión futura
Private Subnet A	Privada	10.0.11.0/24	us-east-1a	Lambda API
Private Subnet B	Privada	10.0.12.0/24	us-east-1b	Lambda Worker, RDS

Cada subnet utiliza un bloque /24, lo cual nos proporciona suficiente espacio de direccionamiento para los recursos contemplados. Para el tamaño actual del proyecto esto es suficiente y además nos deja espacio por si hay que agregar otros componentes en el futuro.

Las subnets públicas tendrán una ruta hacia el Internet Gateway, lo que nos permite que los componentes públicos puedan comunicarse con el internet cuando sea necesario. En esta capa está ubicado el NAT Gateway, debido a que nos permitirá una salida a internet desde las subnets privadas.

Las subnets privadas no tendrán una ruta directa hacia el internet. En su lugar utilizarán el NAT Gateway para poder comunicarse hacia él sin exponerse a riesgos de que cualquiera acceda a ellas. Esto permite que las funciones Lambda o recursos internos puedan acceder a servicios externos o a los servicios administrados por AWS sin tener una IP pública.

13.3 Separación pública y privada

Para esta arquitectura se propone una separación clara entre los componentes expuestos al usuario y los que deben permanecer protegidos dentro de la red. En este caso, los únicos elementos públicos del sistema serán el frontend web y API Gateway, ya que ambos funcionarán como puntos de entrada para las solicitudes de los usuarios.

Las funciones Lambda y la base de datos Amazon RDS MySQL se ubicarán en subnets privadas, sin IP pública. Esta decisión permite que la comunicación pase primero por la capa de aplicación y por los controles definidos mediante security groups. Con ello se reduce la superficie de ataque, se protege la información almacenada y se mantiene aislado el procesamiento interno del sistema.

Dentro del flujo general, API Gateway recibe las solicitudes del frontend y las dirige hacia la Lambda API, encargada de operaciones como la generación de URLs de carga para archivos CSV, la consulta de estados y el historial de reportes. La Lambda Worker funciona como un componente interno dedicado al procesamiento de archivos CSV y a la generación de reportes PDF, mientras que Amazon RDS MySQL permanece protegido en la capa privada y solo acepta conexiones desde los componentes autorizados de la aplicación.

13.4 NAT Gateway vs VPC Endpoints

El equipo consideró la posibilidad de utilizar VPC Endpoints para permitir la comunicación directa con servicios como Amazon S3, sin la necesidad de que el tráfico salga hacia internet. Esta opción resulta atractiva porque mantiene el tráfico dentro de la infraestructura de AWS y puede reducir la dependencia del NAT Gateway para acceder a servicios administrados.

Sin embargo, para esta etapa del proyecto se decidió utilizar NAT Gateway, ya que permite una configuración inicial más sencilla y brinda conectividad saliente general desde las subnets privadas. Esto resulta conveniente mientras el sistema se encuentra en una fase temprana de desarrollo, debido a que las funciones Lambda necesitan acceder a archivos almacenados en Amazon S3 y podrían requerir integración con otros servicios de AWS o dependencias externas a medida que evolucione la solución. De esta forma se evita diseñar múltiples mecanismos de conectividad desde las primeras iteraciones y se mantiene una arquitectura más simple.

La principal desventaja de esta decisión es que el NAT Gateway representa un costo adicional y concentra la salida del tráfico privado. Además, si se implementa únicamente un NAT Gateway, la arquitectura tiene menor tolerancia a fallos en comparación con un diseño que utilice uno por cada Availability Zone.

Este trade-off se considera aceptable debido a que el proyecto tiene un alcance académico, una carga baja y no requiere una configuración productiva de alta disponibilidad. En futuras iteraciones podría evaluarse la incorporación de Gateway Endpoints para Amazon S3, con el fin de reducir el tráfico que pasa por el NAT Gateway, optimizar costos y mejorar la eficiencia de la comunicación dentro de la red de AWS.

14. Procesamiento asíncrono

14.1 Diagrama de contenedores

El diagrama de contenedores se actualiza en base a E3, en esta nueva arquitectura se incluye las colas SQS, la DLQ y el servicio SES. La separación de la red se mantiene sin cambios.

Diagrama 1 – Arquitectura general

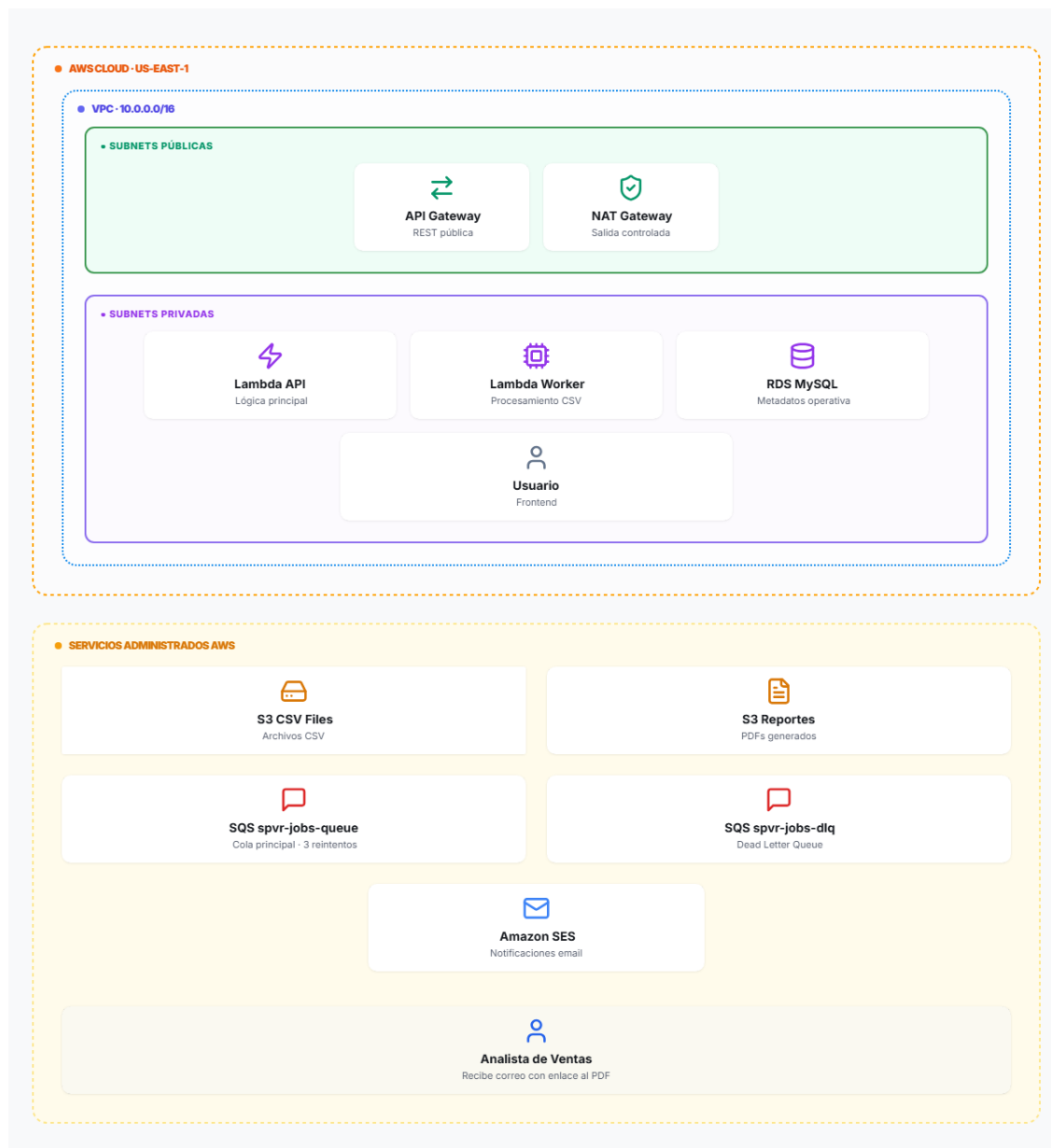


Diagrama 2 – Flujo exitoso

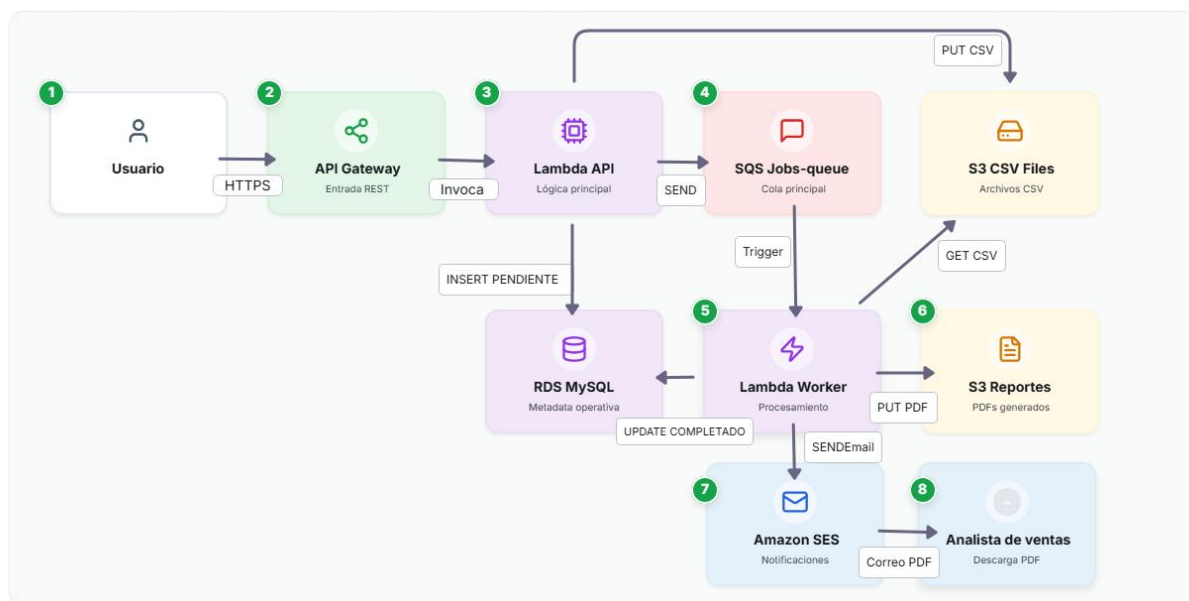


Diagrama 3 – Flujo de error



Tabla de resumen de contenedores

Contenedor / Recurso	Ubicación	Cambio respecto a E3
API Gateway	Capa pública	Sin cambios. Punto de entrada para las solicitudes del frontend.
Lambda API	Subnet privada	Ahora envía un mensaje a la cola SQS spvr-jobs-queue, después registra el trabajo en RDS con estado PENDIENTE y subir el CSV a S3.

Contenedor / Recurso	Ubicación	Cambio respecto a E3
SQS spvr-jobs-queue	Servicio administrado AWS	Es la cola principal, la cual recibe mensajes de Lambda API y los entrega a Lambda Worker.
SQS spvr-jobs-dlq	Servicio administrado AWS	Recibe mensajes que fallaron 3 veces consecutivas y permite que el administrador pueda verificar los trabajos fallidos.
Lambda Worker	Subnet privada	Ahora consume mensajes de la cola SQS, procesa el archivo CSV, genera el PDF y al finalizar llama a SES para notificarle al analista.
Amazon SES	Servicio administrado AWS	Envía el correo de notificación al analista con el enlace al PDF.
RDS MySQL	Subnet privada	No se realizan cambios estructurales, únicamente el campo estado en la tabla trabajos es utilizado para verificar si el procesamiento se completó y evitar que existan duplicados.
S3 CSV Files / S3 Reportes	Servicio administrado AWS	Sin cambios. Lambda Worker lee de S3 el archivo CSV y escribe el reporte PDF generado en S3 Reportes.
NAT Gateway	Subnet pública	Sin cambios. Permite salida desde subnets privadas, sin exponer los recursos.

14.2 Flujos Asíncronos

El sistema SPVR cuenta con dos flujos asíncronos principales. El primer flujo corresponde a un comando, ya que Lambda API le pide a Lambda Worker que realice el procesamiento de un trabajo específico mediante un mensaje a la cola SQS spvr-jobs-queue, en donde se toma y se ejecuta.

El segundo flujo corresponde a un evento, ya que al finalizar el procesamiento Lambda Worker notifica al analista mediante Amazon SES que el reporte generado está listo para poder descargar, sin esperar una respuesta para poder continuar.

En la siguiente tabla se tiene ambos flujos:

Flujo	Productor	Servicio	Consumidor	Tipo	Manejo de fallos
Procesamiento de archivo CSV	Lambda API	SQS	Lambda Worker	Comando	Si falla 3 veces consecutivamente, se envía a DLQ

Flujo	Productor	Servicio	Consumidor	Tipo	Manejo de fallos
					con estado FALLIDO en RDS.
Notificar al analista	Lambda Worker	Amazon SES	Analista de ventas	Evento	En caso de fallo, el analista puede acceder al reporte directamente desde el dashboard consultando el historial de reportes.

14.2.1 Flujo 1 — Procesamiento de archivo CSV (Comando)

El flujo da inicio cuando el analista sube un archivo CSV desde la aplicación. Esto permite que Lambda API pueda registrar el trabajo en RDS y subir el archivo al bucket S3, además podrá enviar el mensaje a la cola SQS, la cual tiene configurado un visibility timeout de 60 segundos para que Lambda Worker pueda completar el procesamiento.

Lambda Worker recibe el mensaje y consulta a RDS el estado del job_id, para garantizar que exista idempotencia, si el estado está en pendiente, lo actualiza a PROCESANDO en donde continua la ejecución para generar el reporte PDF y almacenarlo en S3 marcando el trabajo como COMPLETADO.

En caso de que Lambda Worker falle al momento de realizar el procesamiento, la cola SQS retiene el mensaje y este se reintenta 3 veces de forma consecutiva. Si no cumple después de los 3 intentos el mensaje se enviará a la cola DLQ y el estado del job_id se actualiza a FALLIDO en RDS. Esto permite que el administrador revise el error desde el dashboard.

Formato del payload

Campo	Tipo	Descripción
job_id	Integer	Identificador único del trabajo, permite consultar el estado en RDS.
id_usuario	integer	Usuario que subió el archivo CSV.
csv_s3_key	varchar	Ruta del archivo CSV en el bucket S3 para que Lambda Worker pueda leerlo.
nombre_archivo	varchar	Nombre original del archivo.
fecha_carga	date	Fecha de cuándo se subió el archivo.

14.2.2 Flujo 2 — Notificación al Analista (Evento)

Cuando Lambda Worker finaliza el procesamiento del archivo CSV y el estado del trabajo pasa a COMPLETADO en RDS, procede a llamar a Amazon SES y se envía un correo para notificar al analista que su reporte está listo para descargar.

En este caso, la idempotencia no representa un riesgo, ya que el correo se envía únicamente después de que el estado del trabajo ha sido actualizado a COMPLETADO en RDS. Esto permite asegurar que, por cada trabajo procesado correctamente, se envíe una sola notificación al analista.

Si se presentara un fallo en el envío del correo mediante Amazon SES, el estado del trabajo se mantiene como COMPLETADO ya que el reporte PDF fue procesado, generado y almacenado correctamente en S3.

En este escenario, el analista puede consultar el estado de su trabajo desde el dashboard y descargar directamente el reporte generado. Por lo tanto, un error en la notificación por correo no afecta la disponibilidad del reporte generado ni el resultado del procesamiento.

Formato del payload

Campo	Tipo	Descripción
destinatario	varchar	Correo del analista que subió el archivo CSV.
nombre_archivo	varchar	Nombre del archivo CSV procesado, visible en el correo enviado al analista.
pdf_s3_key	varchar	Ruta del archivo PDF generado en S3 para mostrar el enlace de descarga.
periodo	varchar	Período que cubre el reporte generado.

15. Detalle del componente más complejo

El componente más complejo es Lambda Worker, debido a que la lógica interna se ejecuta principalmente cuando se recibe un mensaje desde SQS. Este componente coordina procesamiento principal del sistema en donde valida el estado del trabajo en RDS, evita procesamientos por medio de idempotencia, obtiene archivos desde S3, calcula métricas de negocio utilizando pandas y genera reportes PDF mediante reportlab.

A diferencia de otros servicios de la arquitectura, como API Gateway, S3 o Amazon SES. Este tiene varias responsabilidades críticas en un solo flujo. Por esta razón, cualquier error en este componente puede afectar directamente la experiencia final del usuario.

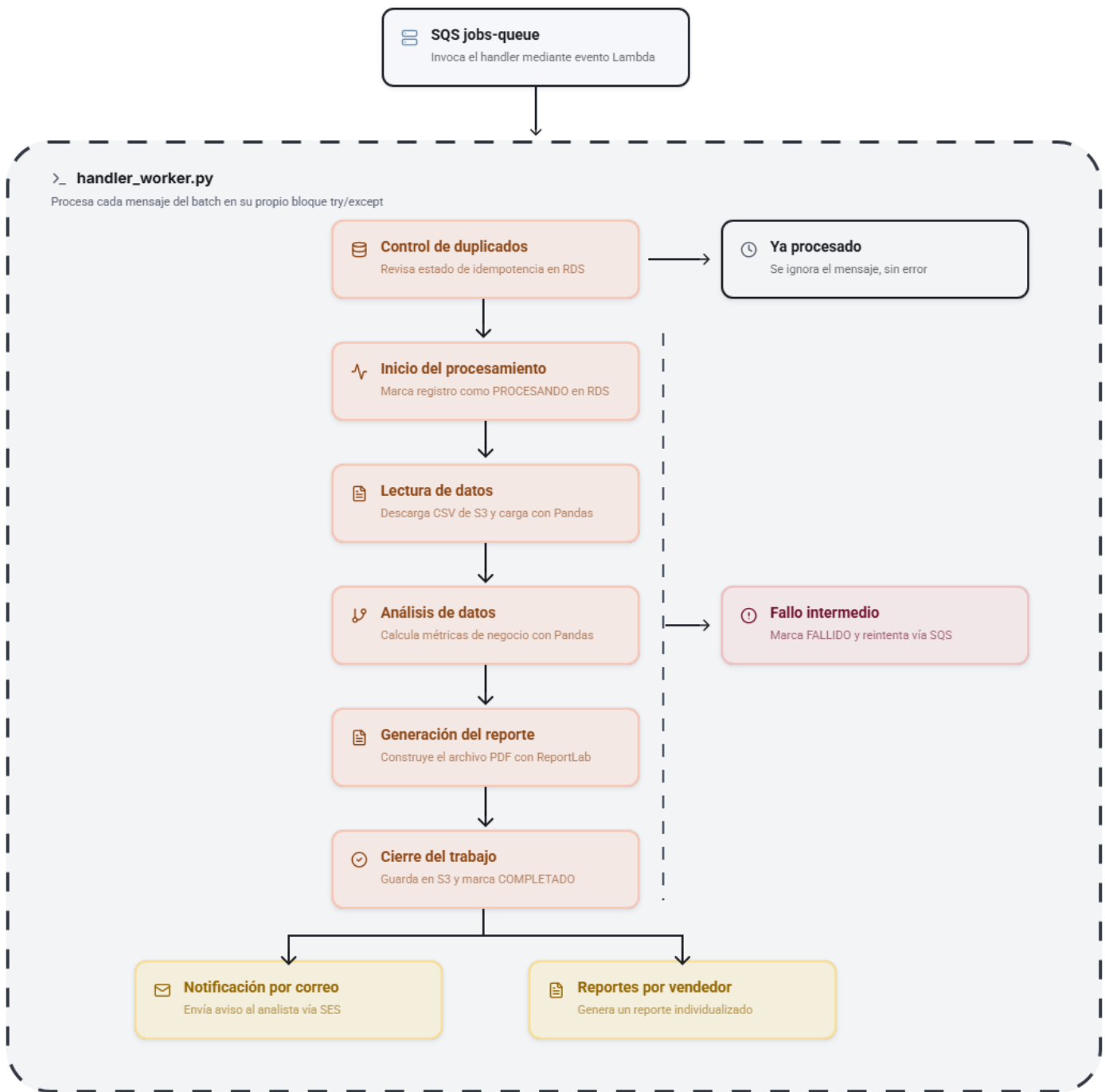


FIGURA 4. COMPONENTES INTERNOS DEL LAMBDA WORKER

Tabla de los componentes Internos

Componente interno de Lambda Worker	Responsabilidad	Consecuencia de Fallo
Control de duplicados	Consulta en el servicio RDS, la tabla trabajo y verifica si el job existe y su estado está PENDIENTE.	Si el job no existe o ha cambiado de estado, los mensajes se descartan con un continue.
Inicio del procesamiento	Actualiza el estado de PENDIENTE en RDS.	El job se marca como FALLIDO y SQS reintenta el mensaje.
Lectura de datos	Descarga el archivo CSV desde S3 y lo carga con pandas.	Si el archivo es inválido o no existe, se marca como FALLIDO y se intenta por medio de SQS.
Análisis de datos	Calcula los totales, ciudades, clientes con pandas y top de productos.	Un error de cálculo (datos corruptos, columnas faltantes o división entre cero) se captura y marca el job como FALLIDO con el detalle del error.
Generación del reporte	El PDF se construye con la herramienta reportlab por medio de sus métricas	Si falla la generación del PDF, el job queda FALLIDO; las métricas ya calculadas no se reutilizan en el reintento.
Cierre del trabajo	Guarda el PDF generado y el JSON de métricas en S3, marcando el job como COMPLETADO, para luego registrar el reporte en RDS.	Si falla a la mitad del proceso, se repite todo el flujo desde el control de duplicados.
Notificación por correo	Se envía el aviso al analista por medio de SES junto con el enlace de descarga, válido únicamente por 30 minutos.	Si SES falla, se registra una advertencia en un log, mientras que el job permanece en COMPLETADO.
Reportes por vendedor	Genera un reporte en S3 y RDS para cada uno de los vendedores.	Si falla el reporte, no afecta a los demás ni al job principal.

15.1 Estados y transiciones

Estado	Quién lo asigna	Transición siguiente
PENDIENTE	Lambda API al crear el trabajo en /upload.	PROCESANDO cuando Worker toma el mensaje.
PROCESANDO	Lambda Worker antes de leer y procesar el CSV.	COMPLETADO si genera reporte; FALLIDO si ocurre error no recuperable.
COMPLETADO	Lambda Worker después de guardar PDF y registrar reporte.	Estado final. El reporte queda disponible para descarga.

FALLIDO	Lambda Worker, Cleanup o administrativa.	Lambda revisión	Estado final operativo; requiere análisis o reproceso manual.
---------	--	-----------------	---

15.2 Dependencias del componente

Dependencia	Uso en el pipeline
RDS MySQL	Fuente de verdad del estado del job, usuarios, reportes y errores.
S3 CSV Files	Almacena archivos CSV cargados mediante URL prefirmada.
SQS spvr-jobs-queue	Desacopla API y Worker, permite reintentos y control de backpressure.
Lambda Worker	Procesa CSV, calcula métricas y genera PDF.
S3 Reportes	Almacena PDF y JSON de métricas.
Amazon SES	Notifica al analista cuando el reporte está listo.
DLQ + Lambda Cleanup	Permiten revisar mensajes fallidos y marcar trabajos estancados.

16. API Surface y escalado

La superficie del API que se documenta a continuación con el verbo, el recurso, el tipo de autenticación esperada, el rol autorizado y los códigos de respuesta principales. En general, los endpoints que acceden a datos de un usuario necesitan un token JWT el cual debe de ser valido. Este token permite determinar qué información puede consultar cada usuario. Las únicas excepciones son el login y la inicialización del entorno, que no requieren sesión porque ocurren antes de que exista una sesión activa.

Endpoint	Autenticación	Rol permitido	Respuesta y errores
POST /login	No requiere	Cualquiera	200 y se entrega el JWT; 400 si falta el correo o la contraseña; 401 si las credenciales son incorrectas.
POST /setup	No requiere (solo disponible en dev/staging)	—	200 al crear las tablas y los datos iniciales; no está expuesto en producción.
POST /upload	JWT	Analista	200 con el job_id , la URL prefirmada para subir el

			archivo,s3_key y status; 401 si no envía un token válido; 400 si el archivo no es un CSV.
POST /jobs/enqueue	JWT	Analista	202 con el message_id de SQS; 400 si no tiene job_id; 401 si no está autenticado; 403 si el job le pertenece a otro usuario; 404 si el job no existe.
GET /jobs	JWT	Analista ve los propios · Admin ve todos	200 con la lista de trabajos según el usuario del token; 401 si no está autenticado.
GET /jobs/{id}	JWT	Dueño del job o Admin	200 con el detalle del job; 403 si no le pertenece; 404 si no existe.
GET /reports	JWT	Analista ve los propios · Gerente y Auditor ven todos	200 con la lista de reportes según el rol; 401 si no está autenticado.
GET /reports/{id}/download	JWT	Dueño del reporte, Gerente o Auditor	200 con la URL temporal del PDF; 403 si no tiene acceso a ese reporte; 404 si no existe.
GET /reports/{id}/metrics	JWT	Dueño del reporte, Gerente o Auditor	200 con el JSON de métricas; mismos errores que el endpoint anterior.
GET /errors	JWT	Administrador	200 con la lista de errores y su referencia a la DLQ; 403 si el usuario no es administrador.
GET /seller/dashboard	JWT	Vendedor (solo su propio panel)	200 con las métricas del vendedor autenticado; 403 si intenta ver el panel de otro vendedor.
GET /seller/history	JWT	Vendedor (solo su propio historial)	200 con su historial de reportes; 401 si no está autenticado.

La tabla anterior muestra el diseño esperado de autenticación y autorización del API. En nuestro diseño, todos los endpoints tienen acceso a los datos del usuario autenticado. La identidad se obtiene exclusivamente a partir de la validación del token JWT, y no mediante ningún parámetro enviado por el cliente. En esta iteración se priorizó completar el flujo principal del sistema: carga de archivos, procesamiento asíncrono, generación de reportes y notificación. La implementación del Lambda Authorizer y la validación de roles quedan como una mejora necesaria antes de usar el sistema fuera de un ambiente de desarrollo.

16.1 Estrategia de escalado y throttling

Para el escalado del sistema se utiliza servicios administrados de AWS, principalmente API Gateway, Lambda, SQS, S3 y RDS. Cada petición HTTP que recibe el API Gateway es ejecutada en una invocación independiente, lo que permite atender solicitudes. Delante del API Gateway se utilizan Amazon CloudFront, principalmente para exponer el dominio personalizado de la API y forzar la redirección de HTTP a HTTPS.

Para evitar la saturación de la API con tareas muy pesadas, se realiza el procesamiento de archivos mediante SQS. Cuando un usuario carga un archivo, Lambda API se encarga de recibirlo, almacenarlo en S3, registrar su data en RDS y enviar un mensaje a la cola. Seguidamente la Lambda Worker consume los mensajes de SQS y procesa los archivos según la concurrencia disponible. De esta forma, varios archivos se pueden generar de manera controlada. Se utiliza S3 para almacenar los archivos CSV y los reportes generados, mientras que RDS almacena la data del proceso, como su estado, fechas y referencias hacia los archivos. Al realizar esto se reduce la carga sobre la base de datos y facilita la escalabilidad en archivos sin tener dependencia directa de sobre RDS cuando se tienen objetos pesados. Actualmente, el endpoint /upload no valida el tamaño ni el tipo del archivo CSV al momento de la carga. Esto implica que un archivo demasiado grande podría aceptarse inicialmente y fallar más adelante durante el procesamiento en Lambda Worker, por ejemplo, al exceder el tiempo máximo de ejecución. Como mitigación futura, si el volumen o tamaño de los archivos supera el rango esperado, se propone mover únicamente Lambda Worker a Fargate, manteniendo Lambda API, SQS y el resto del flujo sin cambios.

Actualmente, no hemos personalizado los límites de tráfico. Tanto la API que recibe peticiones como el sistema que procesa archivos comparten el mismo presupuesto de ejecuciones permitidas por AWS. Por lo que, si recibimos muchas peticiones con archivos pesados, el procesamiento podría consumir toda la capacidad disponible y la API dejaría de responder a los usuarios. Para evitar esto tenemos pensado separar estos recursos limitando los accesos a la API y tendremos una capacidad exclusiva para el procesamiento de archivos.

16.2 Formato de payload

Endpoint	Payload	Campos
POST /login	{ "email": "string", "password": "string" }	Ambos campos son obligatorios. Es el único endpoint donde la identidad del usuario aún no existe.
POST /upload	{ "filename": "string" }	Solo el campo del nombre del archivo es necesario, ya que el dueño del job se asigna a partir del JWT.
POST /jobs/enqueue	{ "job_id": "string" }	Solo el campo que identifica qué trabajo se tiene que encolar es necesario, ya que la validación de a quién le pertenece se hace mediante el token, no contra el body.
GET /jobs/{id}, GET /reports/{id}/download, GET /reports/{id}/metrics	Sin body	El identificador del recurso va en la URL, no en el payload.

Resto de endpoints GET	Sin body	Filtran automáticamente según la identidad y el rol del token; no requieren parámetros adicionales del cliente.
------------------------	----------	---

17. Red Detallada

La red mantiene separación entre capa pública y privada. API Gateway funciona como punto de entrada público, mientras que las funciones Lambda y RDS se mantienen dentro de la capa privada. Los Security Groups permiten que RDS reciba conexiones únicamente desde las funciones que realmente necesitan consultar o actualizar la base de datos.

Capa	Regla permitida	Justificación
API Gateway	Entrada HTTPS desde usuarios.	Expone endpoints controlados para el frontend.
Lambda API	Salida hacia RDS, S3, SQS, Secrets Manager y CloudWatch.	Registra trabajos, genera URLs, autentica usuarios y encola procesamiento.
Lambda Worker	Salida hacia RDS, S3, SQS, SES, Secrets Manager y CloudWatch.	Procesa CSV, guarda reportes, actualiza estados y notifica.
Lambda Cleanup	Salida hacia RDS, DLQ y CloudWatch.	Revisa trabajos estancados o fallidos.
RDS MySQL	Entrada únicamente desde Security Groups de Lambdas autorizadas.	Evita exposición directa de la base de datos a internet.

El tráfico extremo a extremo sigue una ruta única y predefinida. La solicitud del usuario ingresa por HTTPS a través de CloudFront y API Gateway en la capa pública; desde allí, el flujo deriva hacia Lambda API o Lambda Worker en la capa privada. Solo estas funciones Lambda tienen acceso, vía Security Group, a los servicios internos que son: RDS, S3, SQS y SES. De este modo, la capa privada queda completamente protegida, ya que ningún tráfico puede alcanzarla sin haber sido procesado previamente por la capa de aplicación.

18. Modelo de Seguridad

18.1 Roles IAM por servicio

La infraestructura centraliza los roles en un módulo IAM. En lugar de mantener permisos mezclados para varias Lambdas, cada componente cuenta con un rol propio y permisos limitados a los recursos que utiliza.

Rol	Permisos principales	Alcance
lambda_api	Acceso a S3 para archivos, envío de mensajes a SQS, lectura de metadatos y logs propios.	Buckets, cola SQS, RDS y CloudWatch específicos del ambiente.
lambda_worker	Lectura de mensajes SQS, lectura de CSV en S3, escritura de reportes en S3, actualización de estado y logs.	Cola principal, buckets de archivos/reportes, RDS y CloudWatch.
lambda_cleanup	Revisión de trabajos estancados, actualización de estados y escritura de logs.	RDS, DLQ y CloudWatch según lo requerido por el flujo de limpieza.
scheduler_exec	Invocación controlada de Lambda Cleanup.	Únicamente la función Lambda Cleanup.
ci_runner	Permisos de despliegue para Terraform mediante GitHub Actions.	Recursos del proyecto con scoping por servicio cuando AWS lo permite.

Esta separación evita que Lambda API tenga permisos de consumidor de SQS o que Lambda Worker tenga permisos de exposición pública. El rol de CI/CD también se limita al repositorio y al ambiente correspondiente, evitando el uso de credenciales permanentes.

18.2 Secretos gestionados y rotación

La contraseña de RDS ya no se entrega a las Lambdas como variable de entorno en texto plano. En su lugar, las funciones reciben el ARN del secreto mediante DB_SECRET_ARN y consultan AWS Secrets Manager durante el arranque del contenedor Lambda. El valor se mantiene en memoria durante el ciclo de vida de la invocación para evitar lecturas innecesarias.

Secreto	Dueño / consumidor	Rotación y manejo
Contraseña de RDS	Lambda API, Lambda Worker (dueño: equipo de infraestructura)	El secreto está guardado en Secrets Manager con cifrado de KMS, y lo usamos siempre por su identificador ARN , nunca escribiendo el valor directamente en el código.
JWT_SECRET	Lambda API (dueño: equipo de infraestructura)	Por ahora guardamos esta clave como variable de entorno distinta para cada ambiente, pero tenemos pendiente migrarla a Secrets Manager cuando el sistema esté ya en producción para que se renueve automáticamente.

Con esta decisión, la contraseña de base de datos queda almacenada de forma gestionada y cifrada. El pipeline deja de transportar el valor sensible como variable de Terraform o variable de entorno directa de Lambda.

18.3 Autenticación y control de acceso

El sistema implementa autenticación real en lugar de una validación simulada. El login valida credenciales contra hashes bcrypt almacenados en la tabla usuarios y, si son correctas, genera un token JWT firmado con expiración de 8 horas.

bcrypt, JWT y usuarios iniciales

La tabla usuarios incorpora el campo password_hash. Durante la carga inicial de datos, cada usuario de demostración recibe una contraseña temporal y el hash se calcula individualmente dentro del proceso de seed data usando bcrypt. No se almacenan contraseñas en texto plano dentro de la base de datos ni se documentan secretos reales en el diseño.

Para ejecutar la autenticación en Lambda se agregan dos Lambda Layers: una capa para bcrypt y otra para PyJWT. bcrypt requiere empaquetado compatible con Linux porque contiene código compilado, mientras que PyJWT es una dependencia Python pura. El endpoint /login usa bcrypt.checkpw() para validar la contraseña y jwt.encode() para emitir el token.

La variable JWT_SECRET se configura por ambiente y se utiliza para firmar los tokens. En una versión productiva se recomienda mover este valor a Secrets Manager, pero para el diseño se documenta como valor sensible que no debe exponerse en repositorio ni en el documento.

Roles de aplicación

Rol	Permisos funcionales
analista	Carga archivos CSV, consulta el estado de sus trabajos y descarga únicamente los reportes generados por sus propias cargas.
vendedor	Consulta su resumen individual de desempeño sin acceder a datos de otros vendedores.
gerente	Consulta reportes consolidados e históricos para análisis de ventas.
administrador	Gestiona usuarios, revisa errores, DLQ y estado operativo del sistema.
auditor	Consulta reportes en modo de solo lectura para verificar consistencia.

El control de acceso se basa en el rol y en el id_usuario incluido en el token JWT. Cuando un analista consulta su dashboard o historial, Lambda API filtra los trabajos por el id_usuario asociado al token y el campo id_usuario de la tabla trabajos. Esto evita que un analista visualice reportes cargados por otros analistas.

Resultado de pruebas de autenticación

Prueba	Resultado esperado
Login con contraseña correcta	HTTP 200 y token JWT válido con expiración.
Login con contraseña incorrecta	HTTP 401 no autorizado.
Token expirado o inválido	HTTP 401 y rechazo de la solicitud.
Analista consultando reportes ajenos	HTTP 403 o respuesta filtrada sin datos no autorizados.

Se eligió bcrypt + JWT en lugar de Cognito porque permitió incorporar autenticación real sin rediseñar VPC, RDS, API Gateway ni CloudFront. Cognito sigue siendo una alternativa más administrada para producción, pero para el alcance del proyecto la solución actual reduce riesgo de cambios grandes y mantiene seguridad razonable.

18.4 Cifrado con KMS

La arquitectura utiliza una Customer Managed Key por ambiente para proteger los datos almacenados. La llave se referencia mediante un alias por ambiente, por ejemplo, alias/oyd-project-dev-cmk y alias/oyd-project-staging-cmk, evitando documentar identificadores sensibles completos.

Recurso	Uso de KMS
S3 CSV Files	Cifrado de archivos CSV cargados por los analistas.
S3 Reportes	Cifrado de reportes PDF y archivos de métricas generados.
RDS MySQL	Cifrado de almacenamiento de la base de datos.
Secrets Manager	Cifrado del secreto que contiene credenciales sensibles.

La política de la llave limita el uso a los roles Lambda que necesitan descifrar datos, al servicio Secrets Manager y a la administración de la cuenta. No se otorgan permisos abiertos de kms:* ni permisos de descifrado sin condición para todos los principals.

19. Plan de observabilidad

La observabilidad se diseña para responder tres preguntas: si los usuarios pueden usar el API, si el Worker procesa al ritmo esperado y si existen fallos acumulados que requieran intervención. Para esto se utilizan CloudWatch Logs, métricas de API Gateway, métricas de Lambda, métricas de SQS, alarmas y notificaciones por SNS. Los logs de Lambda API y Lambda Worker incluyen el job_id como identificador de correlación en cada línea relevante, lo que permite rastrear un mismo trabajo a través de ambas funciones.

19.1 Logs métricas RED y dashboard

Widget / métrica	Qué mide	Por qué importa
Request count - API Gateway	Cantidad de solicitudes recibidas.	Permite ver demanda real del sistema.
Error rate - Lambda API	Errores del servicio principal.	Indica fallos visibles para el usuario.
Duración - Lambda API / Worker	Tiempo de ejecución por invocación (Duration).	Detecta degradación de desempeño antes de que se convierta en timeout.
Queue depth - SQS	Mensajes visibles pendientes.	Refleja atraso en el procesamiento de reportes.
DLQ messages	Mensajes fallidos retenidos.	Indica trabajos que requieren revisión.

19.2 Alarmas operativas

Alarma	Threshold	Acción	Justificación
lambda-api-errors	5 o más errores en 300 segundos.	Enviar notificación a SNS.	Un patrón repetido de errores puede indicar fallo sistemático de autenticación, RDS o lógica de API.
sqs-queue-depth	50 o más mensajes visibles durante dos	Enviar notificación a SNS.	Una cola alta por 10 minutos indica que Worker no procesa al ritmo de llegada.

	períodos de 300 segundos.		
dlq-messages	Uno o más mensajes en DLQ.	Revisión del administrador.	La DLQ conserva trabajos que no pudieron procesarse y requieren análisis.

Las alarmas no solo registran el evento, también tienen una acción definida: notificar al canal operativo mediante SNS para que el administrador revise el dashboard, la DLQ o los logs relacionados.

19.3 Comportamiento ante degradación

Si RDS deja de estar disponible, Lambda API y Lambda Worker no pueden consultar ni actualizar estados. SQS conserva los mensajes sin perderlos mientras la base de datos se recupera, y CloudWatch registra los fallos de conexión para que el administrador identifique la causa. Una vez restablecida la conexión, los mensajes pendientes se procesan normalmente a través de los reintentos ya configurados, sin necesidad de intervención manual sobre la cola.

20. Costos y presupuesto mensual

El equipo obtuvo el costo real generado por la infraestructura desde Cost Explorer. El monto facturado a la cuenta del equipo es de \$0.00 debido a créditos gratis de AWS aplicados a la cuenta; la siguiente tabla refleja el costo bruto antes de créditos, que es el dato relevante para evaluar la arquitectura, ya que una cuenta sin ese beneficio el cargo sería real.

Servicio	Costo acumulado	Qué lo genera
EC2-Otros	\$14.31	NAT Gateway (cargo por hora + datos procesados)
RDS (Relational Database Service)	\$7.67	Instancia db.t3.micro y almacenamiento de 20 GB.
VPC	\$1.59	Componentes de red asociados a la VPC.
Route 53	\$0.50	Hosted zone del dominio personalizado (api.grupo1.oyd.solid.com.gt).
KMS	\$0.15	Llave administrada por el cliente (CMK) para cifrado de S3, RDS y Secrets Manager.
Secrets Manager	\$0.065	Almacenamiento del secreto de la contraseña de RDS.
S3	\$0.017	Almacenamiento de archivos CSV y reportes PDF.
API Gateway	\$0.0048	Solicitudes HTTP recibidas por Lambda API.
SES	\$0.0032	Envío de notificaciones por correo al analista.
DynamoDB	\$0.0009	Almacenamiento del tstate de la infraestructura.
CloudFront	< \$0.001	Distribución delante de API Gateway.

Total acumulado	\$24.32	
------------------------	----------------	--

El presupuesto de \$20 USD mensuales (AWS Budgets) ya fue superado por el acumulado real de \$24.32. Esto se explica en parte porque el equipo mantiene dos ambientes en paralelo que son: dev y staging, cada uno con su propio NAT Gateway y RDS, lo que duplica los recursos de mayor costo fijo. Se recomienda ajustar el presupuesto a \$25–30 USD, o evaluar si staging necesita estar desplegado de forma continua o solo durante las ventanas de prueba.

21. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Crecimiento de mensajes en SQS	Reportes tardan más en generarse.	Alarma de profundidad de cola y ajuste de concurrencia de Worker.
Mensajes acumulados en DLQ	Trabajos fallidos sin seguimiento.	Alarma, dashboard y revisión por administrador.
Exposición de secretos	Acceso no autorizado a base de datos o tokens.	Secrets Manager, variables sensibles y no documentar secretos reales.
JWT robado o expirado	Acceso indebido si no se valida correctamente.	Expiración de 8 horas, validación en endpoints y futura rotación de secreto.
RDS no disponible	API y Worker no pueden consultar estados.	SQS conserva mensajes y CloudWatch registra fallos para revisión.
CSV demasiado grande	Timeout o consumo excesivo de memoria.	Validar tamaño máximo y considerar mover Worker a Fargate.
Costo de NAT Gateway	Incremento del gasto mensual.	Presupuesto, alerta al 80% y evaluación futura de VPC Endpoints.
El JWT y los roles están diseñados, pero no se validan en tiempo de ejecución.	Un usuario podría ver datos de otro o ejecutar acciones sin tener el rol correspondiente.	Agregar un Lambda Authorizer que valide el JWT y derive la identidad del token, no del cliente.
El endpoint de reseteo no tiene ninguna protección.	Cualquiera con la URL puede borrar toda la base de datos.	Eliminarlo fuera de desarrollo, o protegerlo y restringirlo a administrador.

Destrucción accidental del rol CI	Pipeline pierde capacidad de autenticarse.	prevent_destroy en OIDC Provider y CI Runner Role.
-----------------------------------	--	--

22. Anexo IA — Uso de Inteligencia Artificial en el proyecto completo

22.1 Qué le pedimos a la IA

- Orientación para elegir el tipo de proyecto más conveniente de acuerdo con nuestro nivel de experiencia. Borrador inicial del documento de E1 con la estructura requerida.
- Revisión y mejora del resumen ejecutivo para asegurar coherencia entre el problema, la solución y los beneficios del sistema.
- Definición de los actores del sistema y la diferencia entre actores primarios y de soporte.
- Explicación del rol de cada actor en una empresa real y por qué necesita la información que el sistema genera.
- Sugerencia de las columnas del archivo CSV y de los reportes que el sistema generaría.
- Recomendaciones sobre cómo validar la estructura del CSV mediante una plantilla estándar.
- Validación de la coherencia entre actores, casos de uso, funcionalidades específicas y scope del proyecto.
- Generación de prompts para crear los mockups del frontend y recomendaciones de herramientas gratuitas como Penpot y Figma.
- Diseño del prompt específico para la pantalla de previsualización del reporte PDF desde la perspectiva del vendedor.
- Investigación preliminar sobre los servicios de AWS que podrían utilizarse en el proyecto, sin comprometer una tecnología específica en esa etapa.
- Explicación de la diferencia entre el diagrama de contexto y el diagrama de contenedores, para ubicar correctamente cada componente en su nivel.
- Exploración de las alternativas de cómputo (Lambda, Fargate, EC2) y sus trade-offs para el patrón de carga del sistema.
- Apoyo para estructurar el modelo de datos y definir qué información debe vivir en la base relacional frente a S3.
- Borradores iniciales de los diagramas de contexto y entidad-relación.
- Investigación sobre separación entre subnets públicas y privadas en AWS, y comparación entre NAT Gateway y VPC Endpoints.
- Recomendaciones sobre cómo conectar AWS Lambda con Amazon RDS dentro de una VPC privada y cómo estructurar los security groups entre ambos.
- Explicación de la diferencia entre evento y comando en sistemas asíncronos y cuál aplica a cada flujo del SPVR.
- Comparación entre invocar Lambda Worker directamente desde Lambda API versus usar SQS como intermediario.

- Orientación sobre el visibility timeout en SQS y su relación con el timeout de Lambda Worker.
- Explicación del concepto de idempotencia y cómo implementarlo sin servicios adicionales.
- Alternativas para la notificación al analista entre SES directo y SNS con suscripción SES.
- Justificación del threshold de reintentos y su relación con el costo de Lambda.
- Configuración de AWS Secrets Manager y AWS KMS para el almacenamiento seguro de credenciales.
- Configuración de CloudFront, certificados TLS y dominios personalizados para la publicación segura de la API.
- Integración del frontend con los servicios desplegados en AWS y validación de la comunicación con la API.
- Que revisara si el diagrama y la tabla del componente más complejo cumplían lo que pide la guía, comparándolos directamente contra el código real de handler_worker.py, no contra una versión idealizada.
- Que rehiciera el diagrama del componente más complejo varias veces hasta reflejar el código real, incluyendo el manejo de errores aislado en notificación y reportes por vendedor.
- Que revisara el API surface (handler_api.py) endpoint por endpoint para confirmar si la autenticación JWT documentada realmente se valida en el backend.
- Que calculara un estimado de costos con la calculadora de AWS y lo verificara después contra el gasto real de la cuenta usando Cost Explorer, una vez desplegada la infraestructura, incluyendo identificar a qué corresponde cada línea de la factura.
- Que reorganizara el documento completo en el orden exacto que pide la guía del curso.
- Generación de documentación técnica, evidencias y archivos README para las entregas del proyecto.

22.2 Qué aceptamos y editamos

- El resumen ejecutivo fue generado por IA y ajustado por el equipo hasta reflejar correctamente el problema real.
- Los actores fueron sugeridos por IA, pero el equipo decidió mover al vendedor de actor primario a actor de soporte.
- Las descripciones de los roles fueron simplificadas para representar claramente las funciones de cada actor en una empresa real.
- Las columnas del CSV fueron validadas por el equipo, eliminando unit_cost ya que implicaba calcular márgenes de ganancia fuera del scope.
- Se definió que el sistema utilizará una estructura estándar de columnas para validar los archivos CSV.
- Las user stories y criterios de éxito fueron revisados y ajustados para asegurar consistencia con los actores y el alcance del proyecto.
- Las funcionalidades específicas fueron refinadas para que estuvieran alineadas directamente con los casos de uso priorizados.

- Los mockups fueron generados por IA como borradores y posteriormente rediseñados por el equipo en una herramienta de diseño.
- El criterio de éxito del auditor fue corregido por el equipo, permitiéndole consultar y descargar reportes en modo de solo lectura.
- Se aceptó la decisión de Lambda con los parámetros reales del equipo (512 MB, timeout de 30 s) en lugar de valores genéricos sugeridos.
- Se corrigió la ubicación de S3, Lambda y RDS: la IA aclaró que no son sistemas externos del diagrama de contexto sino componentes internos, lo que el equipo confirmó y aplicó.
- Elegimos RDS MySQL sobre DynamoDB porque el sistema necesita hacer consultas con relaciones, lo cual es natural en SQL.
- El esquema de las cuatro tablas fue revisado y ajustado por el equipo para que los nombres de campos coincidieran con los datos usados.
- Los diagramas finales (contexto y entidad-relación) fueron elaborados por el equipo; la IA solo aportó borradores de referencia.
- Se aceptó la recomendación de separar los componentes públicos y privados dentro de una VPC.
- Se aceptó la recomendación de utilizar dos Availability Zones para distribuir los recursos principales.
- La IA sugirió una única función Lambda para frontend y procesamiento; el equipo decidió separar Lambda API y Lambda Worker para mantener mejor organización.
- Se aceptó usar SQS como intermediario entre Lambda API y Lambda Worker tras validar la tolerancia a fallos con reintentos y DLQ.
- Se aceptó el patrón de idempotencia basado en verificar el campo estado en RDS antes de procesar.
- La distinción evento/comando fue sugerida por la IA y aceptada tras verificar que aplica correctamente a los dos flujos del SPVR.
- Se aceptó reemplazar el diagrama original del componente más complejo por uno que muestra los componentes internos reales de Lambda Worker, tras notar que el primero solo repetía el diagrama de contenedores de E4.
- Se aceptó documentar como riesgo conocido, en lugar de ocultar, que la validación de JWT y de roles está diseñada pero aún no se aplica en tiempo de ejecución, y que el endpoint /admin/reset no tiene ninguna protección.
- Se aceptó reemplazar el estimado teórico de costos por el dato real de Cost Explorer, identificando ahí un cargo de Elastic Load Balancing sin recurso correspondiente en el Terraform, y se ajustó el presupuesto considerando que dev y staging corren en paralelo.
- El equipo revisó y editó el documento completo, redactando las justificaciones de cada decisión con sus propias palabras, de modo que reflejen el razonamiento real del equipo y puedan defenderse en clase.

22.3 Qué descartamos

- El cálculo de comisiones por vendedor, ya que requeriría conocer la estructura de incentivos de cada empresa.

- El campo `unit_cost` en el CSV para calcular márgenes de ganancia, por agregar complejidad innecesaria.
- El filtrado por estado de venta, ya que el archivo CSV será entregado previamente consolidado por el analista.
- La clasificación por severidad en el registro de errores, para mantener el sistema dentro de un alcance manejable.
- El soporte para nombres alternativos de columnas en el CSV, optando por una plantilla estándar obligatoria.
- S3, SQS y SES como actores de soporte, ya que son servicios internos de AWS y no actores externos del sistema.
- La sugerencia inicial de usar DynamoDB para guardar los trabajos, reportes y errores, por no ajustarse a la naturaleza relacional de esos datos.
- Incorporar una capa de caché en esta etapa, por considerarse complejidad innecesaria para el volumen actual.
- La recomendación de utilizar únicamente VPC Endpoints en lugar de NAT Gateway, por simplicidad en esta etapa académica.
- La sugerencia de diseñar alta disponibilidad completa con múltiples NAT Gateway redundantes, por no justificarse para el alcance actual.
- La sugerencia de usar SNS entre Lambda Worker y SES, ya que el SPVR solo tiene un destinatario por trabajo.
- La recomendación de usar EventBridge en lugar de SQS, ya que SQS se ajusta mejor a un único worker consumidor.
- Presentar la tabla de autenticación como si ya estuviera validada en tiempo de ejecución, una vez confirmado en el código que esa validación todavía no existe.
- Dejar el estimado de costos solo como cálculo teórico, una vez que se pudo verificar contra el gasto real de la cuenta.
- Documentar secretos reales, contraseñas, tokens o identificadores sensibles dentro del documento.
- Tratar las respuestas de IA como decisiones finales sin revisión del equipo.

22.4 Reflexión del equipo

El uso de IA ayudó a ordenar el documento, comparar alternativas y revisar coherencia entre entregas. Sin embargo, cada decisión fue ajustada por el equipo para que coincidiera con el funcionamiento real del SPVR y pudiera defenderse técnicamente. La IA sirvió como apoyo para acelerar redacción y validación, no como reemplazo del criterio del equipo. La principal mejora del proyecto fue evolucionar desde una idea funcional hacia una arquitectura completa: primero se definió el producto, luego cómputo, datos y red, después el flujo asíncrono y finalmente seguridad, observabilidad, costos y riesgos. Con esto, el documento final describe qué construir, por qué se eligieron esos servicios y cómo se operaría el sistema en un escenario realista.

Anexo — Mockups del Frontend

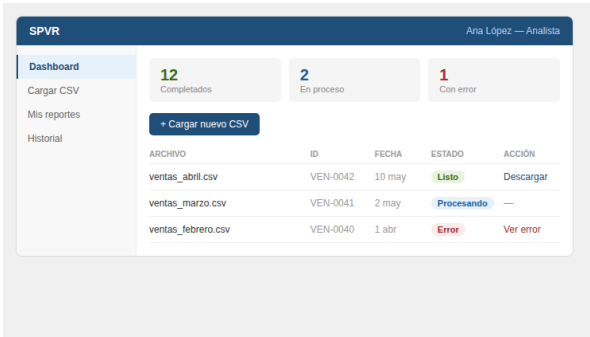
Los siguientes mockups corresponden a las 8 pantallas descritas en la sección 5 de este documento. Se presentan primero los borradores generados con apoyo de inteligencia artificial y posteriormente los mockups finales editados por el equipo.

Borradores

Pantalla 1 — Login



Pantalla 2 — Dashboard principal



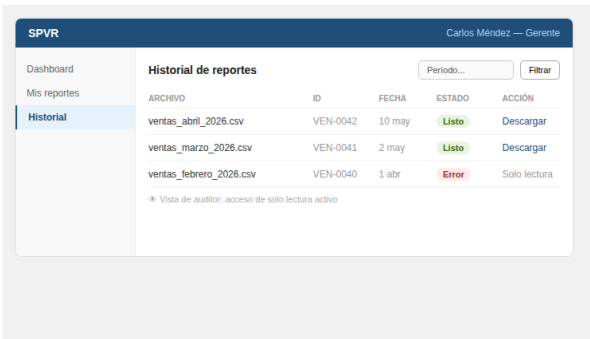
Pantalla 3 — Carga de archivo CSV



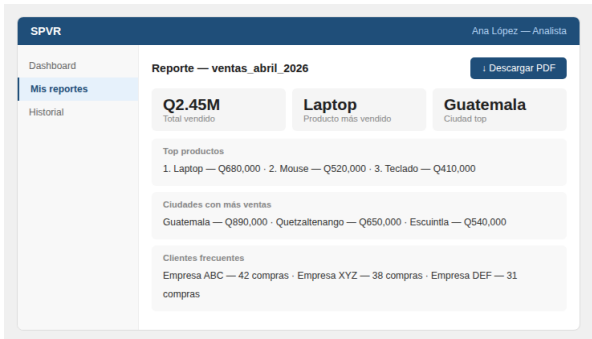
Pantalla 4 — Estado del trabajo en tiempo



Pantalla 5 — Historial de reportes



Pantalla 6 — Previsualización del reporte PDF



Pantalla 7 — Resumen del vendedor



Pantalla 8 — Registro de errores

SPVR Admin — Administrador

Dashboard Usuarios **Registro de errores**

ID TRABAJO	USUARIO	FECHA	DESCRIPCIÓN DEL ERROR
VEN-0040	ana@empresa.com	1 abr	Columna "city" no encontrada en el archivo CSV.
VEN-0038	gerente@empresa.com	15 mar	Tiempo de procesamiento excedido. El archivo supera el límite permitido.
VEN-0035	ana@empresa.com	3 mar	Error al generar el PDF: datos insuficientes en columna "quantity".

Mockups Finales

Pantalla 1 — Login

SPVR

Iniciar Sesión

Accede al Sistema de Procesamiento de Ventas y Reportes

Correo electrónico

ejemplo@empresa.com

Contraseña [¿Olvidaste tu contraseña?](#)

.....

Ingresar

Al ingresar, aceptas nuestros términos de servicio y políticas de privacidad.

¿No tienes cuenta? Contacta con tu [administrador de sistema](#)

© 2026 SPVR - v1.0.0

[Privacidad](#) [Soporte Técnico](#) [Documentación](#)

Pantalla 2 — Dashboard principal



DashboardCargar CSVHistorial

🔔

Ana López
Analista

➔

¡Hola de nuevo, Ana!

Aquí tienes un resumen de la actividad de procesamiento de hoy.

Cargar nuevo CSV

Reportes Generados

24

+4 desde ayer



En Procesamiento

3

Tiempo prom. 2.5 min



Con Error

1

Requiere revisión manual



Mis trabajos recientes

Ver todo el historial

Archivo	ID Único	Fecha de Carga	Estado	Acciones
 ventas_mayo_2026.csv	A-0024	24 May 2026, 14:30	PROCESANDO	 Ver estado ...
 reporte_anual_2025_v2.csv	A-0023	23 May 2026, 09:15	COMPLETADO	 Descargar ...
 ventas_noreste_final.csv	A-0022	22 May 2026, 18:45	COMPLETADO	 Descargar ...
 error_carga_datos.csv	A-0021	22 May 2026, 11:20	FALLIDO	 Ver error ...
 consolidado_ventas_q1.csv	A-0020	21 May 2026, 10:00	COMPLETADO	 Descargar ...

© 2026 SPVR - v1.0.0

PrivacidadSoporte TécnicoDocumentación

Pantalla 3 — Carga de archivo CSV

SPVR

Dashboard

Cargar CSV

Historial

Ana López
Analista

🔔

Columnas requeridas

Asegúrate de que tu archivo CSV contenga exactamente los siguientes encabezados para un procesamiento exitoso.

📄

sale_id

📄

sale_date

📄

product_name

📄

quantity

📄

unit_price

📄

city

📄

status

📄

salesperson_name

📄

customer_id

📄

customer_name

* El orden de las columnas no es relevante, pero los nombres deben coincidir exactamente.

VALIDACIÓN DEL SISTEMA

🔔

Error de Validación

Falta la columna 'status' en el archivo cargado. Por favor, verifica el archivo y vuelve a intentarlo.

Cargar archivo CSV de ventas

Sube tus datos de ventas para iniciar el procesamiento y generación de reportes.

Arrastra tu archivo CSV aquí o haz clic para seleccionar

Solo se admiten archivos en formato .csv

Procesar Archivo >

Al procesar, el sistema validará la estructura y tipo de datos.

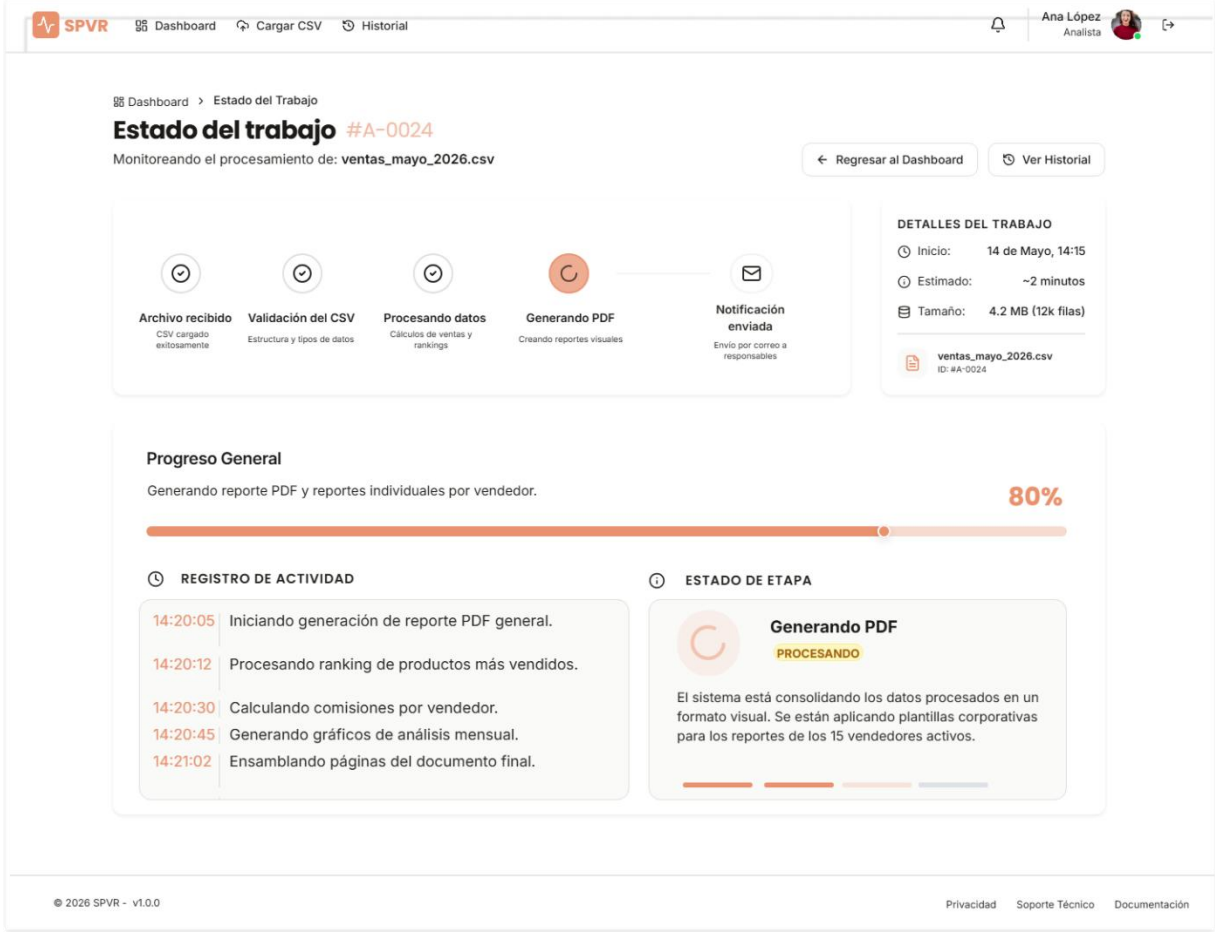
© 2026 SPVR - v1.0.0

Privacidad

Soporte Técnico

Documentación

Pantalla 4 — Estado del trabajo en tiempo real



Pantalla 5 — Historial de reportes

SPVR

DashboardCargar CSVHistorial

Diego Perez

Gerente

Dashboard > Historial

Historial de Reportes

Últimos 30 días

Consulta y gestiona todos los trabajos de procesamiento realizados en el sistema.

TOTAL PROCESADOS
1,284

EN ESPERA
12

ERRORES CRÍTICOS
3

Búsqueda

Periodo

Mayo 2026

▼

🔍

Q Buscar por nombre de archivo...

Archivo	Usuario	Fecha de Creación	Estado	Acciones
 ventas_mensuales_mayo_2026.csv CSV • 2.4 MB	ana@	15 Mayo 2026, 14:30	COMPLETADO	 Descargar  Detalles ...
 reporte_anual_consolidado.csv CSV • 15.8 MB	ana@	14 Mayo 2026, 09:15	COMPLETADO	 Descargar  Detalles ...
 ventas_sucursal_norte_v2.csv CSV • 1.1 MB	luis@	14 Mayo 2026, 18:45	FALLIDO	 Descargar  Detalles ...
 procesamiento_masivo_clientes.csv CSV • 45.2 MB	ana@	13 Mayo 2026, 11:20	PROCESANDO	 Descargar  Detalles ...
 ventas_abril_final_v3.csv CSV • 3.7 MB	luis@	12 Mayo 2026, 16:05	COMPLETADO	 Descargar  Detalles ...

Mostrando 1-5 de 24 reportes

< 1 2 3 >

© 2026 SPVR - v1.0.0

PrivacidadSoporte TécnicoDocumentación

Pantalla 6 — Previsualización del reporte PDF





DashboardHistorialErrores



Admin Principal
Administrador IT



Registro de Errores

Monitorea y diagnostica fallos en el procesamiento de ventas y generación de reportes.

Errores Hoy

12

+4 desde ayer

Pendientes

5

Errores sin revisar

Filtrar Resultados

Q

Buscar por ID de trabajo, nombre de usuario o fecha de incidencia...

Incidentes Recientes

Visualización detallada de los últimos 50 errores detectados

Actualizado hace 2 min

ID Error	ID Trabajo	Usuario	Fecha y Hora		Descripción del Error
#ERR-99241	JOB-A0024	 Ana López ANALISTA	16 May 2026, 14:32		Error de validación en columna 'customer_id'. Tipo de dato no coincide con el esquema CSV.
#ERR-99240	JOB-A0022	 Carlos Pérez ANALISTA	16 May 2026, 12:15		Fallo en la conexión con el servidor de generación de PDF. Tiempo de espera agotado.
#ERR-99238	JOB-A0019	 Luis Porras ANALISTA	15 May 2026, 09:45		Error de lectura de archivo: el archivo 'ventas_q1.csv' está dañado o vacío.
#ERR-99235	JOB-A0015	 Roberto Gómez ANALISTA	14 May 2026, 18:20		Formato de fecha incorrecto en columna 'sale_date'. Se esperaba DD/MM/YYYY.
#ERR-99230	JOB-A0012	 Ana López ANALISTA	14 May 2026, 11:05		Símbolo de moneda no reconocido en campo 'unit_price'. Se esperaba 'Q'.

Mostrando 1 a 5 de 124 errores

< Anterior

1

2

3

...

25

Siguiente >

© 2026 SPVR - v1.0.0.

PrivacidadSoporte TécnicoTérminos de Servicio